
UNIT 10 CREATING RESTFUL APIS IN PYTHON

- 10.0 Introduction
 - 10.1 Objectives
 - 10.2 Basic Concepts
 - 10.3 Installing Required Libraries
 - 10.4 Building RESTful APIs with Flask
 - 10.5 Building RESTful APIs with FastAPI
 - 10.6 Summary
 - 10.7 Solution/Answers
 - 10.8 References/ Further Readings
-

10.0 INTRODUCTION

In today's digital world, everything is becoming more connected and automated. Applications are no longer isolated; they often need to communicate with other apps, websites, and services over the internet. For example, a food delivery app needs to talk to Google Maps to show the delivery location, a weather app needs to fetch live data from a server, and a shopping app might need to process payments using a third-party service. But how do these apps "talk" to each other? This is made possible through APIs, which stands for Application Programming Interfaces.

An **API** acts like a bridge that allows two software systems to exchange data and interact with each other. A very common and widely used type of API is called a **RESTful API**. REST stands for **Representational State Transfer**. It is a set of simple rules that allow computers and applications to communicate over the web. Among various types of APIs, RESTful APIs have become the most widely used because they are simple, scalable, and easy to implement.

Python offers several good tools and frameworks for building RESTful APIs. In this unit, we will focus on two widely used frameworks: **Flask** and **FastAPI**. Both Flask and FastAPI help developers build APIs that work across different platforms — like Android apps, iOS apps, web browsers, and even IoT devices — regardless of the programming language being used on the other side. Once you create a RESTful API, many clients can use it. This makes the application more efficient, organised, and scalable.

Today, some of the biggest services we use every day — like Google Maps, Twitter, YouTube, and GitHub — provide RESTful APIs so that other applications can connect to them and use their features.

This unit introduces you to the fundamentals of creating RESTful APIs using Python. In this unit, you will learn how APIs work, what REST means, and how to build real-world APIs using two powerful Python frameworks: Flask and FastAPI. These tools allow developers to design web services that are clean, efficient, and ready to integrate into larger systems.

Whether you are building a student project, a web application, or a backend for a mobile app, understanding how to create and use RESTful APIs will be a valuable skill. By the end of this unit, you will be able to build your own APIs, test them, and understand how they fit into the larger software ecosystem.

10.1 OBJECTIVES

After completing this unit, you will be able to:

- Understand what RESTful APIs are, why they are important, and how they help different applications communicate over the web,
- Identify the key parts of REST architecture, such as:
 - Resources (the data you work with),
 - Endpoints (URLs used to access data),
 - HTTP methods like GET, POST, PUT, DELETE (used to request or change data).
- Set up the Python development environment by installing required tools and libraries like Flask and FastAPI,
- Build RESTful APIs using Flask, and learn how to handle different types of HTTP requests and return proper responses,
- Create APIs using FastAPI, and explore advanced features such as:
 - Automatic data validation,
 - Built-in documentation,
 - Support for asynchronous code for faster processing.
- Work with JSON data to send input to the API and receive output from the API,
- Perform CRUD operations — Create, Read, Update, Delete — through RESTful API endpoints,
- Connect your API to a database, so that the data can be stored permanently and retrieved when needed, and
- Test the functionality of your APIs, and learn how to add error handling and validate user input to make your APIs reliable and user-friendly.

10.2 BASIC CONCEPTS

Before we begin coding RESTful APIs in Python, it is important to understand the key concepts that form the building blocks of API development. These ideas may sound technical at first, but once you grasp them, building APIs will become much easier and more intuitive.

10.2.1 What is an API?

An API, or Application Programming Interface, is a set of rules that allows two software applications to communicate with each other. You can think of it as a translator or bridge between different programs. These programs do not need to know how the other works on the inside. When you use an app on your phone to book a cab, it talks to a server somewhere to get details like car availability, driver location, or estimated fare—this exchange of information happens through an API.

More technically, an API defines the methods and data formats that applications should use to request and exchange data. For example, an API may allow one app to send a question like "What's the weather in Delhi today?" and get a well-structured response like:

```
{"city": "Delhi", "temperature": "32°C", "status": "Sunny"}
```

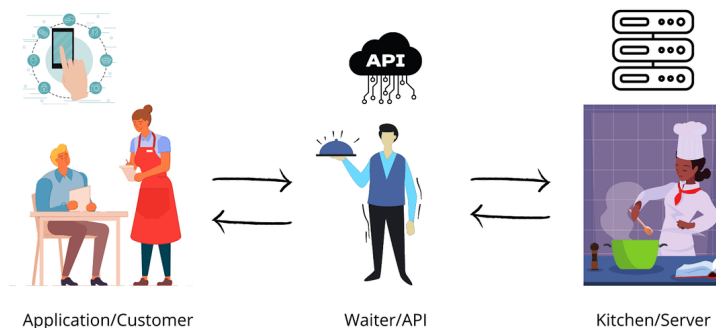


Figure: 10.1 Real-life analogy demonstrating the role of an API

Imagine you go to a restaurant (refer to Figure 10.1). You (the customer) want to order food, but you do not go into the kitchen. Instead, you tell the waiter what you want. The waiter then communicates your order to the kitchen (the server), and brings the food (the response) back to you. Just like the waiter understands both the customer and the kitchen, the API knows how to handle the communication between your app and the backend.

APIs allow developers to:

- Access specific features or data of a service (like Google Maps, Twitter, or a payment system).
- Request information or send updates without directly connecting to the system's database.
- Build smarter, faster applications by using ready-made services.

APIs are not limited to web services—they are also used in desktop software, games, operating systems, and more.

10.2.2 Web API and HTTP

Web APIs are APIs that are accessible using standard web protocols, most commonly HTTP (Hypertext Transfer Protocol). HTTP is the protocol that browsers use to access web pages, and Web APIs use the same protocol to send and receive data.

When a client (like a browser or mobile app) wants to interact with a Web API, it sends an HTTP request to the server, and the server responds with an HTTP response. These responses usually include data in JSON format (which we will learn more about soon) and a status code indicating whether the request was successful or not.

Web APIs are commonly used to allow apps to send and receive data from servers. For example:

- Google Maps API

- Twitter API
- Weather API
- Google Translate API

These APIs return data in formats like **JSON** or **XML**.

10.2.3 Example - Google Translate API

If you want to translate a text, you may use the services of Google Translate. You either type or copy-paste a text in a source language. Then you select the destination language, and the translated text is generated. A sample Google Translate snapshot is shown in *Figure 10.2*.

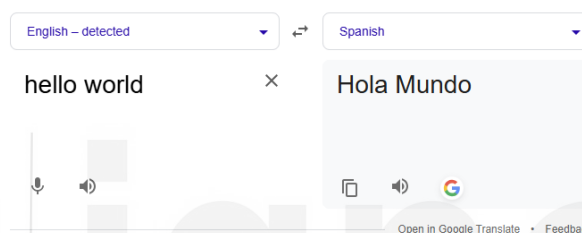


Figure: 10.2 Snapshot of Google Translate

However, this works only for short text or paragraphs. When you need to translate a huge dataset spanning over multiple text files (say 2098 files), it becomes tedious to open each file, copy the text, translate it and keep it for further processing. This is where the Google Translate API comes in to help.

Google Translate is a REST-based API offered by Google Cloud that lets developers programmatically translate text between languages using HTTP requests.

Given below is an example for posting request to translate the English text into Spanish.

```
POST
https://translation.googleapis.com/language/translate/v2?key=YOUR_API_KEY1

Content-Type: application/json

{
  "q": "Hello world",
  "source": "en",
  "target": "es",
  "format": "text"
}
```

¹ To obtain a Google Translate API key, you need to create a project in the Google Cloud Console and enable the Cloud Translation API. Then, you generate an API key within the Credentials section of your project.

The received output is:

```
{
  "data": {
    "translations": [
      {
        "translatedText": "Hola mundo"
      }
    ]
  }
}

>>> Translated Text: Hola mundo
```

10.2.3. What is REST?

REST (Representational State Transfer) is an architectural style for designing networked applications. It was introduced by Roy Fielding in his PhD thesis in 2000. REST is not a standard, but rather a set of guiding principles that make web services easy to use, scalable, and stateless.

REST is a set of rules or principles used to design web services, especially Web APIs. It helps developers build applications that can communicate easily over the internet using standard web protocols like HTTP.

A RESTful API follows these six main constraints:

- * **Statelessness:** Every request from a client must contain all the information needed to process it. The server does not store anything about the user's session.
- * **Client-Server Architecture:** The client (e.g., a mobile app) and the server (where the data is stored) must be separate. This separation allows each to be developed independently.
- * **Uniform Interface:** All RESTful APIs must follow a consistent method of interacting with resources, usually using standard HTTP methods.
- * **Resource-Based:** Everything the API manages (like users, products, or orders) is treated as a resource. Each resource is accessible via a URL.
- * **Representation of Resources:** Resources are usually sent in JSON or XML formats. Clients interact with these representations, not the raw data.
- * **Cacheable:** Responses must indicate whether they can be cached to improve performance.

10.2.6 API Request Structure

Understanding how **requests** and **responses** work in RESTful APIs is crucial before building your own API. Every time a client (such as a web app or mobile app) wants to interact with a server through an API, it sends a **request**. The server processes this request and sends back a **response**.

When a client requests a RESTful API, it includes:

- The **HTTP method** (e.g., GET or POST)
- The **URL** identifying the resource (e.g., /products/5)

- Optional **headers** (e.g., content type, authentication tokens)
- Optional **body** (used with POST, PUT, or PATCH to send data)

a) HTTP Method

This tells the server what type of action the client wants to perform. The common methods are:

- **GET** – Ask the server for some data (e.g., fetch a list of products).
- **POST** – Send new data to the server (e.g., add a new user).
- **PUT** – Update existing data entirely (e.g., change all details of a product).
- **PATCH** – Update part of the data (e.g., change only the price of a product).
- **DELETE** – Remove a resource from the server (e.g., delete a comment).

b) URL (Uniform Resource Locator)

The **URL** points to the specific resource the client wants. For example:

```
GET /products/5
```

This request asks the server to return the product with ID 5.

c) Headers

Headers provide **metadata**—extra information about the request. Some common headers include:

- **Content-Type: application/json** – Tells the server that the request body is in JSON format.
- **Authorization: Bearer <token>** – Provides a security token so that only authorized users can access the resource.
- **Accept: application/json** – Tells the server the client wants the response in JSON format.

d) Request Body

Some types of requests (especially **POST**, **PUT**, and **PATCH**) include a **body**. This is where the client sends the actual data to the server. For instance, to create a new user, the request body might look like this:

```
{  
  "name": "Alice",  
  "email": "alice@example.com"  
}
```

The server uses this data to perform the desired operation, such as saving a new user to the database.

10.2.10 What is Included in an API Response?

After the server receives and processes the client's request, it sends back a **response**. The response also has several key parts:

The server processes the request and sends back a **response** that includes:

- An **HTTP status code** (e.g., 200 OK, 201 Created, 404 Not Found)
- A **response body**, usually in **JSON** format, containing data or confirmation
- Response **headers**

a) HTTP Status Code

This code tells the client what happened during the request. Some common status codes are:

- 200 OK – The request was successful.
- 201 Created – A new resource was successfully created.
- 400 Bad Request – Something was wrong with the request sent by the client.
- 401 Unauthorized – The client needs to log in or provide valid credentials.
- 404 Not Found – The resource was not found.
- 500 Internal Server Error – There was a problem on the server's side.

b) Response Headers

Like request headers, **response headers** provide additional information.

Examples:

- Content-Type: application/json – Indicates the format of the returned data.
- Cache-Control – Tells the browser how long to store (cache) the data.
- Set-Cookie – Sends a cookie to the client to maintain session data.

c) Response Body

This is the main part of the response and usually contains **JSON data**. The contents depend on what was requested. For example:

```
{
  "id": 5,
  "product_name": "Wireless Mouse",
  "price": 499
}
```

If the request was to delete a product, the body might just contain a message:

```
{
  "message": "Product deleted successfully"
}
```

Understanding this **request-response structure** will help you debug API issues, write client-side code, and design more reliable APIs.

10.2.11 Data Format: JSON

RESTful APIs most commonly use **JSON (JavaScript Object Notation)** for data exchange because it is lightweight, human-readable, and easy to parse. In Python, JSON can be handled easily using built-in libraries. Data sent in JSON format is used both for sending input (requests) and receiving output (responses).

A typical JSON object, describing a user using three key-value pairs, might look like:

```
{
  "name": "Rahul",
  "email": "rahul@example.com",
  "isActive": true
}
```

By mastering these fundamental concepts, you will be well-prepared to build real-world RESTful APIs using Python. In the next sections, we will explore how to install the necessary tools and begin developing APIs using two powerful frameworks: Flask and FastAPI.

10.2.12 CRUD operations in REST APIs

In most applications, data needs to be created, read, updated, and deleted. These four fundamental operations are collectively known as CRUD:

- C – Create: Adding a new resource (e.g., registering a new student).
- R – Read: Retrieving existing data (e.g., viewing the list of students).
- U – Update: Modifying an existing resource (e.g., changing a student’s course).
- D – Delete: Removing a resource (e.g., deleting a student record).

Table 10.1: CRUD Operations

CRUD Operation	HTTP Method	Example Endpoint	Description
Create	POST	/students	Add a new student to the system
Read (all)	GET	/students	Retrieve the list of students
Read (one)	GET	/students/{id}	Retrieve details of a specific student
Update	PUT	/students/{id}	Update information of a student
Delete	DELETE	/students/{id}	Remove a student record

Check your progress 1

1. What is an API, and how does it help two applications communicate? Explain using a real-world example.

2. Describe the structure of an HTTP request and explain each component using an example.

3. Suppose you are building a mobile app to show weather updates. Explain how RESTful API concepts like HTTP methods, JSON data format, and client-server interaction work together to display the weather data to the user.

4. REST APIs often use standard HTTP methods. Match the following operations with the correct HTTP method:

- (a) Retrieve a resource
- (b) Create a new resource
- (c) Update an existing resource
- (d) Delete a resource

5. Critically analyze the role of statelessness in REST architecture. What issues might arise if APIs were not stateless? Give practical examples.

6. Design a RESTful API workflow for a railway reservation system. Identify possible endpoints, HTTP methods, and example request/response structures.

10.3 INSTALLING REQUIRED LIBRARIES

To build RESTful APIs in Python, we need certain external libraries and frameworks that simplify the process of creating web applications and handling HTTP requests and responses. As mentioned earlier in this unit, we will work with two popular Python frameworks — **Flask** and **FastAPI**. Before using them, you need to ensure that your system has Python installed and also, the required libraries are properly set up.

Checking Python Installation

Before proceeding, make sure Python is installed on your system. Open your terminal or command prompt and type:

```
python--version
```

, or on some systems, it can be

```
python3 --version
```

Installing pip

pip is Python's package manager. It is usually included with Python installations. You can check if it is available by running:

```
pip --version or pip3 --version
```

Installing Flask

Flask is a lightweight web framework ideal for beginners. To install Flask using pip, type:

```
pip install Flask
```

You can verify the installation by checking the version:

```
python -m flask --version
```

This will confirm that Flask is correctly installed in your environment.

Installing FastAPI and Uvicorn

FastAPI is a modern web framework that is fast, intuitive, and supports asynchronous programming. It also automatically generates API documentation.

To begin using FastAPI, we need to install two libraries: fastapi and uvicorn.

Fastapi is the main library to build the API and **uvicorn** is the ASGI server used to run the application.

To install FastAPI along with **Uvicorn** (an ASGI server required to run FastAPI applications), use the following command:

```
pip install fastapi uvicorn
```

FastAPI: The framework itself.

Uvicorn: A fast ASGI server to run your app.

To test the installation, you can run a basic FastAPI server.

Other Useful Tools

- **requests** – A simple library for making HTTP requests (useful for testing APIs from Python)

```
pip install requests
```

- **httpie** – A user-friendly command-line tool to test APIs (alternative to curl)

```
pip install httpie
```

With these installations complete, you are ready to start building and running RESTful APIs using Python. In the following sections, we will learn how to use these libraries to create functional API endpoints.

10.4 BUILDING RESTFUL APIS WITH FLASK

Flask is a lightweight and beginner-friendly web framework for Python. It helps you build web applications and APIs quickly. Flask is ideal for small projects or learning purposes because it is simple, flexible, and has minimal setup. With Flask, we can create a server, define the paths (or URLs) it will respond to, and decide what to do when someone visits those paths.

In this section, we will learn how to create RESTful APIs using Flask. We will cover how to define API routes, handle HTTP methods, and send and receive JSON data.

10.4.1 Creating a Simple API with Flask

Let us create a very basic RESTful API using Flask. This API will allow us to perform operations on a list of books.

Step 1: Basic Flask App Structure

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Welcome to the Book API!"
if __name__ == '__main__':
    app.run(debug=True)
```

The code works as follows:

- We import the Flask library and create an app using `Flask(__name__)`.
- We define a simple route `/`, which is like a homepage of API using `@app.route('/')`.
- `home()` function runs when someone visits the root path (`/`). When someone visits this path, it shows the message *"Welcome to the Flask API!"*.
- Running this code will start a server on your local computer (usually at `http://localhost:5000/` or <http://127.0.0.1:5000/>) (unless you change the port).

10.4.2 Returning Data as JSON

In REST APIs, we usually send and receive data in **JSON format**.

```
from flask import jsonify

@app.route('/books', methods=['GET'])
def get_books():
    books = [
        {"id": 1, "title": "Python Basics"},
        {"id": 2, "title": "Flask for Beginners"}
    ]
    return jsonify(books)
```

- `@app.route('/books', methods=['GET'])`: Defines a **GET endpoint** at `/books`.
- `jsonify(books)`: Converts the Python list to JSON format automatically.

10.4.3 Adding a New Book (POST Method)

Let us now create a **POST endpoint** to add a new book.

```
from flask import request
books = []
@app.route('/books', methods=['POST'])
def add_book():
    new_book = request.get_json()
    books.append(new_book)
    return jsonify({"message": "Book added successfully"}), 201
```

- `request.get_json()`: Reads the JSON body sent by the client.
- `books.append(new_book)`: Adds the book to the list.
- `201`: This is the status code for “Created”.

Example Input (JSON):

```
{
  "id": 3,
  "title": "Learn REST APIs"
}
```

10.4.4 Getting a Single Book (GET with ID)

We can fetch a specific book using its ID from the URL:

```
@app.route('/books/<int:book_id>', methods=['GET'])
def get_book(book_id):
    for book in books:
        if book['id'] == book_id:
            return jsonify(book)
    return jsonify({"error": "Book not found"}), 404
```

- `<int:book_id>`: This tells Flask to expect an integer in the URL.
- If the book is found, we return it. Otherwise, we return a 404 error.

10.4.5 Updating a Book (PUT Method)

We can also update an existing book:

```
@app.route('/books/<int:book_id>', methods=['PUT'])
def update_book(book_id):
    updated_data = request.get_json()
    for book in books:
        if book['id'] == book_id:
            book.update(updated_data)
            return jsonify(book)
    return jsonify({"error": "Book not found"}), 404
```

- PUT replaces the existing book data with the new data.
- book.update (updated_data): Merges the new values into the existing dictionary.

10.4.6 Deleting a Book (DELETE Method)

Finally, let us allow users to delete a book

```
@app.route('/books/<int:book_id>', methods=['DELETE'])
def delete_book(book_id):
    for book in books:
        if book['id'] == book_id:
            books.remove(book)
            return jsonify({"message": "Book deleted"})
    return jsonify({"error": "Book not found"}), 404
```

- books.remove(book): Deletes the book from the list.
- Responds with a confirmation message.

10.4.7 Complete Example: Book API

Here's a full example combining all the above endpoints into one complete app:

```
from flask import Flask, request, jsonify

app = Flask(__name__)

books = [
    {"id": 1, "title": "Python Basics"},
    {"id": 2, "title": "Flask for Beginners"}
]

@app.route('/')
def home():
    return "Welcome to the Book API!"

@app.route('/books', methods=['GET'])
def get_books():
    return jsonify(books)

@app.route('/books', methods=['POST'])
def add_book():
    new_book = request.get_json()
    books.append(new_book)
    return jsonify({"message": "Book added successfully"}), 201

@app.route('/books/<int:book_id>', methods=['GET'])
def get_book(book_id):
    for book in books:
        if book['id'] == book_id:
            return jsonify(book)
    return jsonify({"error": "Book not found"}), 404
```

```
@app.route('/books/<int:book_id>', methods=['PUT'])
def update_book(book_id):
    updated_data = request.get_json()
    for book in books:
        if book['id'] == book_id:
            book.update(updated_data)
            return jsonify(book)
    return jsonify({"error": "Book not found"}), 404

@app.route('/books/<int:book_id>', methods=['DELETE'])
def delete_book(book_id):
    for book in books:
        if book['id'] == book_id:
            books.remove(book)
            return jsonify({"message": "Book deleted"})
    return jsonify({"error": "Book not found"}), 404

if __name__ == '__main__':
    app.run(debug=True)
```

10.4.8 Invoke the Book API

Get all the books

Request	Response
GET http://localhost:5000/books	Full list of books in JSON format <pre>[{ "id": 1, "title": "Python Basics" }, { "id": 2, "title": "Flask for Beginners" }]</pre>

Add a new book

Request	Response
POST http://localhost:5000/books Content-Type: application/json Body: <pre>{ "id": 3, "title": "API Development in Python" }</pre>	<pre>{ "message": "Book added successfully" }</pre>

Update a book

Request	Response
<pre>PUT http://localhost:5000/books/3 Content-Type: application/json Body: { "title": "Advanced API Development" }</pre>	<pre>{ "id": 3, "title": "Advanced API Development" }</pre>

10.4.9 Advanced Example: Student Management API with Token Auth, SQLite, Validation & Pagination

Here, you will learn to build a secure API to manage students in a university system. Each student has id, name, and course. The API must expose CRUD endpoints, protect them with a Bearer token, validate inputs, paginate results, and store data persistently in SQLite. Assume the SQLite Data model includes the Students table. The Students table schema is given below:

Students(id INTEGER PRIMARY KEY, name TEXT NOT NULL, course TEXT NOT NULL)

Step 1: Project Setup and Configuration

```
from flask import Flask, request, jsonify, g
import sqlite3
from functools import wraps

app = Flask(__name__)
app.config.update(
    DATABASE="students.db",
    API_TOKEN="mysecrettoken123", # demo token (replace with env var in prod)
    JSON_SORT_KEYS=False         # keep response fields in insertion order
)
```

In the code snippet above, **g** stores request-scoped objects (like the DB connection). For teaching/learning purposes, we have hardcoded the API token here. In production, we generally use variables or a vault.

Step 2: Database helpers (open/close per request)

```
def get_db():
    db = getattr(g, "_db", None)
    if db is None:
        db = g._db = sqlite3.connect(app.config["DATABASE"])
        db.row_factory = sqlite3.Row
    return db

@app.teardown_appcontext
def close_db(exc):
    db = getattr(g, "_db", None)
    if db is not None:
        db.close()
```

```
# Initialize schema once
with sqlite3.connect(app.config["DATABASE"]) as conn:
    conn.execute("""
        CREATE TABLE IF NOT EXISTS students (
            id INTEGER PRIMARY KEY,
            name TEXT NOT NULL,
            course TEXT NOT NULL
        )
    """)
```

Here, the `row_factory` lets us access columns by name for clean JSON conversion.

Step 3: Centralized error handling

```
@app.errorhandler(404)
def not_found(_):
    return jsonify({"error": "Resource not found"}), 404

@app.errorhandler(400)
def bad_request(_):
    return jsonify({"error": "Bad request"}), 400

@app.errorhandler(401)
def unauthorized(_):
    return jsonify({"error": "Unauthorized"}), 401

@app.errorhandler(500)
def server_error(_):
    return jsonify({"error": "Internal server error"}), 500
```

Step 4: Token authentication decorator

```
def require_token(f):
    @wraps(f)
    def wrapper(*args, **kwargs):
        token = request.headers.get("Authorization", "")
        if token != f"Bearer {app.config['API_TOKEN']}":
            return jsonify({"error": "Unauthorized"}), 401
        return f(*args, **kwargs)
    return wrapper
```

Step 5: Validate Utilities

```
def validate_student_payload(payload, require_all=True):
    if not isinstance(payload, dict):
        return "JSON body must be an object"
    name = payload.get("name")
    course = payload.get("course")

    if require_all:
        if not name or not course:
```

```

    return "Fields 'name' and 'course' are required"
else:
    if name is None and course is None:
        return "At least one of 'name' or 'course' is required"

    if name is not None and not str(name).strip():
        return "'name' cannot be empty"
    if course is not None and not str(course).strip():
        return "'course' cannot be empty"
    return None

```

Step 6: Routes

```

# List with pagination
@app.route("/students", methods=["GET"])
def list_students():
    try:
        offset = int(request.args.get("offset", 0))
        limit = min(int(request.args.get("limit", 10)), 100) # safety cap
    except ValueError:
        return jsonify({"error": "offset and limit must be integers"}), 400

    cur = get_db().cursor()
    cur.execute("SELECT id, name, course FROM students LIMIT ? OFFSET ?", (limit, offset))
    rows = [dict(r) for r in cur.fetchall()]
    return jsonify({
        "data": rows,
        "pagination": {"offset": offset, "limit": limit, "count": len(rows)}
    })

# Get one
@app.route("/students/<int:sid>", methods=["GET"])
def get_student(sid):
    cur = get_db().cursor()
    cur.execute("SELECT id, name, course FROM students WHERE id = ?", (sid,))
    row = cur.fetchone()
    if not row:
        return jsonify({"error": "Resource not found"}), 404
    return jsonify(dict(row))

# Create (requires token)
@app.route("/students", methods=["POST"])
@require_token
def create_student():
    payload = request.get_json(silent=True) or {}
    err = validate_student_payload(payload, require_all=True)
    if err:
        return jsonify({"error": err}), 400

    cur = get_db().cursor()
    cur.execute("INSERT INTO students(name, course) VALUES(?, ?)",

```

```

        (payload["name"].strip(), payload["course"].strip()))
    get_db().commit()

    sid = cur.lastrowid
    return jsonify({"id": sid, "name": payload["name"].strip(),
                  "course": payload["course"].strip()}), 201

# Update (merge semantics; requires token)
@app.route("/students/<int:sid>", methods=["PUT"])
@require_token
def update_student(sid):
    payload = request.get_json(silent=True) or {}
    err = validate_student_payload(payload, require_all=False)
    if err:
        return jsonify({"error": err}), 400

    db = get_db()
    cur = db.cursor()
    cur.execute("SELECT id, name, course FROM students WHERE id = ?",
                (sid,))
    row = cur.fetchone()
    if not row:
        return jsonify({"error": "Resource not found"}), 404

    new_name = payload.get("name", row["name"]).strip()
    new_course = payload.get("course", row["course"]).strip()
    cur.execute("UPDATE students SET name = ?, course = ? WHERE id = ?",
                (new_name, new_course, sid))
    db.commit()
    return jsonify({"id": sid, "name": new_name, "course": new_course})

# Delete (requires token)
@app.route("/students/<int:sid>", methods=["DELETE"])
@require_token
def delete_student(sid):
    db = get_db()
    cur = db.cursor()
    cur.execute("DELETE FROM students WHERE id = ?", (sid,))
    if cur.rowcount == 0:
        return jsonify({"error": "Resource not found"}), 404
    db.commit()
    return jsonify({"message": "deleted"})

```

Step 7: Testing the API

(POST)		
curl -X POST http://localhost:5000/students \ -H "Content-Type: application/json" \ -H "Authorization: Bearer mysecrettoken123" \ -d '{"name":"Aisha","course":"Data Science"}'	{"id": 1, "name": "Aisha", "course": "Data Science"}	201 created
List with Pagination		
curl "http://localhost:5000/students?offset=0&limit=2"	{"data":[{"id":1,"name":"Aisha","course":"Data Science"}],	200 OK

	"pagination":{"offset":0,"limit":2,"count":1}}	
localhost:5000/students/1 \	{"id":1,"name":"Aisha","course":"AI & ML"}	200 OK
: application/json" \		
: Bearer mysecrettoken123" \		
& ML"}'		
tp://localhost:5000/students/1 \	{"message":"deleted"}	200 OK
: Bearer mysecrettoken123"		

Check your progress 2

1. Explain the difference between a GET and POST request.
.....
.....
2. Create a Flask app with two endpoints:
 - o /hello: Returns "Hello, World!" using GET method.
 - o /goodbye: Returns "Goodbye!" using GET method.

.....
3. Create an API to return the square of a number sent via a query parameter.
Example: URL: /square?num=5
 Response: {"result": 25}

4. Explain what happens when a request is sent to an undefined endpoint (like /unknown).
.....
.....
5. Extend the Book API to support searching for books by title keyword. What endpoint and method will you use? Write the sample Flask code.
.....
.....
6. Consider an API that processes large image files. Discuss how you would handle file uploads with Flask while ensuring performance and security.
.....
.....
7. Modify the Book API so that it validates incoming data when adding a new book. The API should reject requests where the title is missing or empty. Write the sample Flask code.
.....
.....
8. APIs should not expose sensitive error details to clients. How would you handle errors in Flask using a global error handler? Write example code for handling 404 and 500 errors.
.....
.....

10.5 BUILDING RESTFUL APIS WITH FASTAPI

Now that you have learned how to create RESTful APIs using Flask, let us explore another modern, efficient, and developer-friendly framework—**FastAPI**.

FastAPI is like Flask's younger sibling —faster, modern, and packed with features that make building APIs easy and enjoyable.

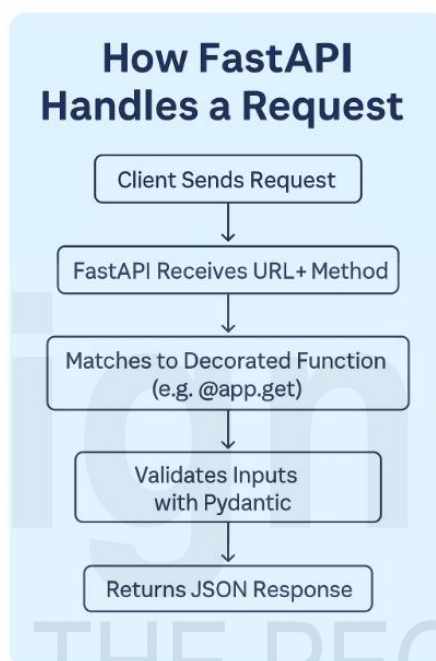


Figure 2: FAST API Request Processing

Now let us learn how to create APIs using FastAPI.

10.5.1 What is FastAPI?

FastAPI is a modern, high-performance Python framework for building APIs quickly and efficiently. It is designed to create RESTful APIs that are fast, easy to use, and production-ready. Unlike Flask, FastAPI is built on top of **ASGI** (Asynchronous Server Gateway Interface), which means it supports asynchronous programming, allowing for faster and more scalable applications.

One key feature of the FastAPI is that it uses **Python type hints** to automatically validate input data and generate interactive API documentation (Swagger UI). This saves development time and helps developers avoid common mistakes.

FastAPI is built on top of **Starlette** (for the web parts) and **Pydantic** (for data validation) and offers speed, automatic documentation, and strong validation with minimal effort.

FastAPI is fast — both in development and execution. It provides automatic interactive API documentation, type checking with Python hints, and data validation using Pydantic models.

The reasons of popularity of FastAPI are:

Speed: As the name suggests, FastAPI is fast — both in development and execution.

Automatic Documentation: It provides you with beautiful docs (Swagger UI and Redoc) out of the box.

Type Safety: You get error checking while writing code, thanks to Python type hints.

Validation: It automatically checks input data using Pydantic models.

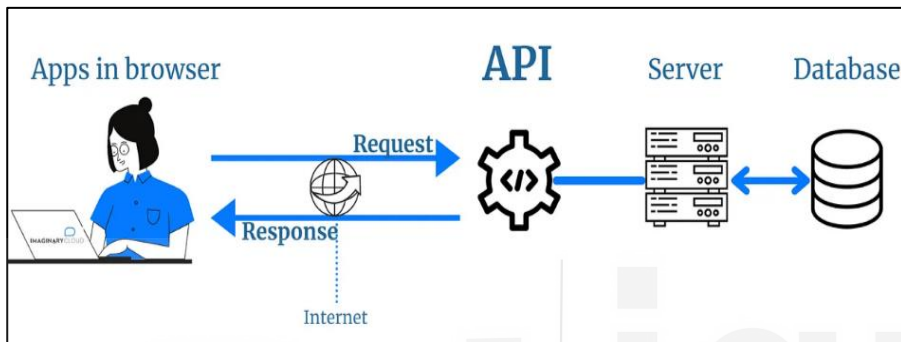


Figure 3: Request/ Response Model

10.5.2 Creating Your First FastAPI App

Here is a basic FastAPI app:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to FastAPI!"}
```

- FastAPI() creates your app.
- @app.get("/") tells FastAPI that this function will run when a **GET** request is made to the root (/) URL.
- It returns a simple JSON message.

Run this app using:

```
uvicorn myapp:app --reload
```

Then open your browser and go to <http://localhost:8000/> to see the result.

Also check <http://localhost:8000/docs> — you will find your API documentation ready.

To run the server, use the command:

```
uvicorn main:app --reload
```

When this runs, your API will be available at <http://127.0.0.1:8000/>, and you can view auto-generated documentation at:

- <http://127.0.0.1:8000/docs> (Swagger UI)
- <http://127.0.0.1:8000/redoc> (ReDoc UI)

This built-in documentation lets developers test APIs directly in the browser.

10.5.3 Adding Routes: GET, POST, DELETE

Let us build a **Book API**, similar to our Flask example discussed earlier in this unit, but with FastAPI.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# In-memory book storage
books = []

# Define the Book schema using Pydantic
class Book(BaseModel):
    id: int
    title: str

@app.post("/books")
def add_book(book: Book):
    books.append(book)
    return {"message": "Book added successfully"}

@app.get("/books")
def get_books():
    return books

@app.delete("/books/{book_id}")
def delete_book(book_id: int):
    for book in books:
        if book.id == book_id:
            books.remove(book)
            return {"message": "Book deleted"}
    return {"error": "Book not found"}
```

10.5.4 Example Inputs and Outputs

POST /books

Request:

```
{
  "id": 1,
  "title": "Learning FastAPI"
}
```

Response:

```
{
  "message": "Book added successfully"
}
```

GET /books

Response:

```
[
  {
    "id": 1,
    "title": "Learning FastAPI"
  }
]
```

DELETE /books/1

Response:

```
{
  "message": "Book deleted"
}
```

10.5.5 Protecting Your API (Simple Example)

```
from fastapi import FastAPI, Request

app = FastAPI()

@app.get("/secure")
def secure_route(token: str = ""):
    if token != "admin123":
        return {"error": "Access Denied"}
    return {"message": "Welcome Admin!"}
```

Try with URL:

<http://localhost:8000/secure?token=admin123>

10.5.6 Advanced Example: Library Catalog API with Async Endpoints, Pydantic Validation, Nested Models & Background Tasks

Design an API to manage a library catalog of books and authors. Each book has id, title, year, and one author (with name, optional country). Provide full CRUD for books, keyword search with pagination/sorting, input validation, and a background task to simulate post-create processing (e.g., indexing or sending a notification).

Pydantic Data Models

```
from pydantic import BaseModel, Field
from typing import Optional

class Author(BaseModel):
    name: str = Field(..., min_length=2, max_length=80)
    country: Optional[str] = Field(default=None, max_length=80)

class Book(BaseModel):
    id: int = Field(..., ge=1)
    title: str = Field(..., min_length=3, max_length=120)
    year: int = Field(..., ge=1500, le=2100)
    author: Author
```

- The annotated full implementation is given below:

```
from fastapi import FastAPI, HTTPException, Query, BackgroundTasks,
Depends, status
from pydantic import BaseModel, Field
from typing import List, Optional
import asyncio

app = FastAPI(title="Library Catalog API", version="1.0")

# ---- Models ----
class Author(BaseModel):
    name: str = Field(..., min_length=2, max_length=80)
    country: Optional[str] = Field(default=None, max_length=80)

class Book(BaseModel):
    id: int = Field(..., ge=1)
    title: str = Field(..., min_length=3, max_length=120)
    year: int = Field(..., ge=1500, le=2100)
    author: Author

class BookUpdate(BaseModel):
    title: Optional[str] = Field(default=None, min_length=3,
max_length=120)
    year: Optional[int] = Field(default=None, ge=1500, le=2100)
    author: Optional[Author] = None

# In-memory store (replace with DB in production)
BOOKS: List[Book] = []

# ---- Dependencies (e.g., auth) ----
def require_api_key(x_api_key: Optional[str] = Query(default=None,
alias="api_key")):
    # simple demo auth via query param; replace with OAuth2/JWT later
    if x_api_key != "demo-key":
        raise
    HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid API key")
    return True
```

```
# ---- Helpers ----
async def fake_db_latency():
    await asyncio.sleep(0.2)

async def index_book_background(book: Book):
    # simulate indexing or notification
    await asyncio.sleep(0.1)
    # log/print omitted here; assume success

# ---- Routes ----
@app.get("/books", response_model=List[Book], summary="List or search books")
async def list_books(
    keyword: Optional[str] = Query(default=None, description="Search in title/author"),
    skip: int = Query(default=0, ge=0),
    limit: int = Query(default=10, ge=1, le=100),
    sort: Optional[str] = Query(default="title", description="Sort by 'title' or 'year'"),
):
    await fake_db_latency()
    items = BOOKS
    if keyword:
        k = keyword.lower()
        items = [b for b in items if k in b.title.lower() or k in b.author.name.lower()]

    if sort == "year":
        items = sorted(items, key=lambda b: b.year)
    else:
        items = sorted(items, key=lambda b: b.title.lower())

    return items[skip: skip + limit]

@app.get("/books/{book_id}", response_model=Book, summary="Get one book by ID")
async def get_book(book_id: int):
    await fake_db_latency()
    for b in BOOKS:
        if b.id == book_id:
            return b
    raise HTTPException(status_code=404, detail="Book not found")

@app.post("/books", response_model=Book, status_code=201, summary="Create a new book")
async def create_book(book: Book, bg: BackgroundTasks, ok: bool = Depends(require_api_key)):
    await fake_db_latency()
    if any(b.id == book.id for b in BOOKS):
        raise HTTPException(status_code=400, detail="ID already exists")
    BOOKS.append(book)
```

```

bg.add_task(index_book_background, book) # non-blocking post-create
work
return book

@app.put("/books/{book_id}", response_model=Book, summary="Update
book (merge)")
async def update_book(book_id: int, patch: BookUpdate, ok: bool =
Depends(require_api_key)):
    await fake_db_latency()
    for i, b in enumerate(BOOKS):
        if b.id == book_id:
            updated = b.copy(update=patch.dict(exclude_unset=True))
            BOOKS[i] = updated
            return updated
    raise HTTPException(status_code=404, detail="Book not found")

@app.delete("/books/{book_id}", summary="Delete book")
async def delete_book(book_id: int, ok: bool = Depends(require_api_key)):
    await fake_db_latency()
    for i, b in enumerate(BOOKS):
        if b.id == book_id:
            BOOKS.pop(i)
            return {"message": "deleted"}
    raise HTTPException(status_code=404, detail="Book not found")

```

Testing the API calls

Create (with API)	curl -X POST "http://localhost:8000/books?api_key=demo-key" \ \ -H "Content-Type: application/json" \ -d '{ "id": 101, "title": "Deep Learning", "year": 2016, "author": {"name": "Ian Goodfellow", "country": "USA"} }'
Search, Paginate, Sort	curl "http://localhost:8000/books?keyword=deep&skip=0&limit=5&sort=year"
Update (merge)	curl -X PUT "http://localhost:8000/books/101?api_key=demo-key" \ -H "Content-Type: application/json" \ -d '{"year": 2018}'
Delete	curl -X DELETE "http://localhost:8000/books/101?api_key=demo-key"

Check your progress 3

1. What is FastAPI, and why is it popular for building APIs?

.....

.....

2. What Python package do we need to install for using FastAPI?

3. Explain the role of `@app.get()` and `@app.post()` decorators in FastAPI. Give examples.

4. What is the use of pydantic models in FastAPI? Give a basic example.

5. Design and implement a RESTful API using Flask for managing a simple “Student Records” system. The API should support the following operations:

1. GET /students → Retrieve all students.
2. GET /students/<id> → Retrieve a single student by ID.
3. POST /students → Add a new student.
4. PUT /students/<id> → Update an existing student’s details.
5. DELETE /students/<id> → Delete a student record.

Each student record should contain:

- id (integer)
- name (string)
- course (string)

Write the Flask code for this API and demonstrate with example requests and responses.

6. Your API currently stores data in a Python list. Discuss the limitations of this approach and redesign the API to use a database (e.g., SQLite). Write sample Flask code to fetch all books from the database.

10.6 SUMMARY

This unit explained how to create RESTful APIs using Python. RESTful APIs help different applications or systems talk to each other using simple web-based rules (HTTP methods). We explored two popular Python frameworks — **Flask** and **FastAPI**—to build and test these APIs. Both frameworks allow developers to send and receive data over the web in an organized and structured way, mostly using JSON format. This unit also guided us through the process of installing libraries, creating endpoints, handling data, and comparing the strengths of each framework.

10.7 SOLUTION TO CHECK YOUR PROGRESS

Answer to Check your progress -1

1. An API (Application Programming Interface) is a set of rules that allows two software applications to talk to each other. It acts like a bridge or translator between two programs.

For example, when you use a cab booking app like Ola or Uber, the app needs to get information such as:

- Available cars nearby,
- Estimated arrival time,
- Fare details.

This information is not stored in your mobile app; it is fetched from the company's servers. The app sends a request through an API, and the server sends a response. You see the result in real time on your screen.

2. Every time a client (such as a mobile app or browser) wants to talk to a RESTful API, it sends an HTTP request. The main parts of this request are: HTTP method, URL, Headers, and request body.

3. The weather data to the user is displayed by following the following steps:

- Client Sends a Request - The user opens a weather app. The app (client) sends an HTTP GET request to a weather API
- Server Processes the Request - The server receives this request. It looks up weather data for "Delhi" from its database or another source and then prepares a response.
- Server Sends a Response - The response is sent back in JSON format
- Client Displays Data - The weather app receives the JSON data

4. (a) Retrieve a resource → GET

(b) Create a new resource → POST

(c) Update an existing resource → PUT (or PATCH for partial update)

(d) Delete a resource → DELETE

5. Role of Statelessness:

- Each request from the client must contain all information needed.
- The server does not store client session data.
- This improves scalability, reliability, and load balancing.

If not stateless (stateful API):

- Server would need to store session info for every client.
- Increased memory usage, risk of crashes.
- Harder to scale horizontally (across multiple servers).

Practical Example:

If a railway booking API was stateful, switching servers during load balancing would break the booking process. Stateless design avoids this problem.

6. In a railway reservation system, typical resources include:

- **Trains** (information about available trains)
- **Stations** (source and destination)
- **Reservations** (ticket booking)
- **Passengers** (traveller details)
- **Payments** (booking confirmation)
- **Endpoints & Methods**
 - GET /trains → List all available trains
 - GET /trains/{id} → Get details of a specific train
 - POST /bookings → Create a new booking
 - GET /bookings/{id} → Get booking details
 - DELETE /bookings/{id} → Cancel a booking

Answer to Check your progress -2

1. The differences between a GET and POST request are:

GET Request	POST Request
Used to retrieve data from the server.	Used to send data to the server (usually to create or update something).
Data is sent through the URL (as query parameters).	Data is sent in the body of the request.
Data is visible in the browser address bar (e.g., /search?name=python).	Data is not visible in the URL. It is hidden in the request body.
Example of searching for a book title.	Submitting a new book to the system.
It can be repeated without side effects.	No – repeating a POST can create duplicate data.

2. The Flask app with two endpoints:

```
from flask import Flask

app = Flask(__name__)

# GET endpoint to say Hello
@app.route('/hello', methods=['GET'])
def say_hello():
    return "Hello, World!"

# GET endpoint to say Goodbye
@app.route('/goodbye', methods=['GET'])
def say_goodbye():
    return "Goodbye!"

if __name__ == '__main__':
    app.run(debug=True)
```

3. An API to return the square of a number sent via a query parameter.

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/square', methods=['GET'])
def calculate_square():
    # Get the query parameter "num"
    num = request.args.get('num', default=0, type=int)
    result = num * num
    return jsonify({"result": result})

if __name__ == '__main__':
    app.run(debug=True)
```

Example usage: <http://localhost:5000/square?num=5>

Response: { "result": 25 }

4. When you access an endpoint that **‘does not exist’** in your Flask app, Flask returns an **HTTP 404 Not Found** error.

5. To search books by a keyword in their title, we can create a **GET endpoint** like:

GET /books/search?keyword=python

Here, keyword is passed as a query parameter. Flask provides request.args.get() to extract query parameters.

```
@app.route('/books/search', methods=['GET'])
def search_books():
    keyword = request.args.get('keyword', "").lower()
    results = [book for book in books if keyword in book['title'].lower()]

    if results:
        return jsonify(results)
    return jsonify({"message": "No books found with given keyword"}), 404
```

Request: GET <http://localhost:5000/books/search?keyword=flask>

Response:

```
[
  { "id": 2, "title": "Flask for Beginners" }
]
```

6. Handling large file uploads requires careful consideration of both performance and security.

Steps for Implementation:

Step 1. Enable File Uploads in Flask:

Use `request.files` to accept uploaded files.

```
from flask import Flask, request

app = Flask(__name__)
app.config['UPLOAD_FOLDER'] = 'uploads/'
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024 # 16 MB limit

@app.route('/upload', methods=['POST'])
def upload_file():
    if 'file' not in request.files:
        return {"error": "No file part"}, 400
    file = request.files['file']
    if file.filename == '':
        return {"error": "No selected file"}, 400
    file.save(app.config['UPLOAD_FOLDER'] + file.filename)
    return {"message": "File uploaded successfully"}, 201
```

Step 2: Performance Considerations:

- Use streaming uploads for very large files instead of loading them fully into memory.
- Store files in a dedicated file server or cloud storage (e.g., AWS S3, Google Cloud Storage).
- Process images asynchronously with background workers (e.g., Celery, RQ) to avoid blocking API responses.

Step 3. Security Considerations:

- Limit maximum file size using `MAX_CONTENT_LENGTH`.
- Validate file types (e.g., allow only .jpg, .png).
- Sanitize file names using libraries like `werkzeug.utils.secure_filename()`.
- Store uploaded files outside the web root to prevent direct access.

7. We can add simple input validation inside the `add_book()` endpoint.

```
@app.route('/books', methods=['POST'])
def add_book():
    new_book = request.get_json()

    # Validation: check if 'title' exists and is not empty
    if not new_book or 'title' not in new_book or new_book['title'].strip() == '':
        return jsonify({"error": "Book title is required"}), 400

    books.append(new_book)
    return jsonify({"message": "Book added successfully"}), 201
```

Examples:

Request	Response	Remarks
{	{	Invalid

<code>"id": 4 }</code>	<code>"error": "Book title is required" }</code>	
<code>{ "id": 5, "title": "Machine Learning with Python" }</code>	<code>{ "message": "Book added successfully" }</code>	Valid

8. In Flask, we can define global error handlers using `@app.errorhandler`.

```
@app.errorhandler(404)
def not_found_error(error):
    return jsonify({"error": "Resource not found"}), 404

@app.errorhandler(500)
def internal_error(error):
    return jsonify({"error": "Internal Server Error. Please try again later."}), 500
```

- `@app.errorhandler(404)` → Catches all "Not Found" errors and returns a JSON response.
- `@app.errorhandler(500)` → Catches unexpected server errors without exposing internal stack traces.
- This improves **security** (no stack trace leaks) and **user experience** (consistent JSON error format).

Example Response for 404:

```
{  
  "error": "Resource not found"  
}
```

Example response for 500:

```
{  
  "error": " Internal Server Error. Please try again later."  
}
```

Answer to Check your progress -3

1. FastAPI is a modern, fast (high-performance) web framework for building APIs with Python 3.6+. FastAPI is asynchronous and uses modern Python features like type hints and automatic validation with Pydantic. It also provides auto-generated documentation and faster request handling with Uvicorn.

2. To use FastAPI, you need to install two main packages –

- `fastapi`: The framework to build the API.
- `uvicorn`: A lightning-fast ASGI server to run your FastAPI app.

```
pip install fastapi
pip install "uvicorn[standard]"
```

3. In FastAPI, `@app.get()` and `@app.post()` are decorators used to define HTTP endpoints.

- `@app.get("/route")`: Used when the client wants to get (retrieve) data. For example, in the code below, visiting `/hello` in the browser (GET request) returns: `{"message": "Hello, world!"}`
- `@app.post("/route")`: Used when the client wants to send (submit) data to the server. Sending data using POST to `/submit` returns: `{"received": {your data}}`

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/hello")
def say_hello():
    return {"message": "Hello, world!"}

@app.post("/submit")
def receive_data(data: dict):
    return {"received": data}
```

4. Pydantic models are used to define and validate the data structure of requests and responses.

- Validates incoming data automatically
- Converts JSON to Python objects
- Returns error if data is invalid

Consider the code given below. Here, `Book` is a Pydantic model. It expects `title` (string), `author` (string), and `pages` (integer). If the user sends wrong data (e.g., `pages` as a string), FastAPI will return a validation error.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Book(BaseModel):
    title: str
    author: str
    pages: int

@app.post("/addbook")
def add_book(book: Book):
    return {"message": "Book received", "title": book.title}
```

5.

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# Sample in-memory data
```

```
students = [
    {"id": 1, "name": "Alice", "course": "Computer Science"},
    {"id": 2, "name": "Bob", "course": "Mathematics"}
]

# Home route
@app.route('/')
def home():
    return "Welcome to the Student Records API!"

# 1. Get all students
@app.route('/students', methods=['GET'])
def get_students():
    return jsonify(students)

# 2. Get a single student by ID
@app.route('/students/<int:student_id>', methods=['GET'])
def get_student(student_id):
    for student in students:
        if student['id'] == student_id:
            return jsonify(student)
    return jsonify({"error": "Student not found"}), 404

# 3. Add a new student
@app.route('/students', methods=['POST'])
def add_student():
    new_student = request.get_json()
    if not new_student or "id" not in new_student or "name" not in
new_student or "course" not in new_student:
        return jsonify({"error": "Invalid data"}), 400
    students.append(new_student)
    return jsonify({"message": "Student added successfully"}), 201

# 4. Update an existing student
@app.route('/students/<int:student_id>', methods=['PUT'])
def update_student(student_id):
    updated_data = request.get_json()
    for student in students:
        if student['id'] == student_id:
            student.update(updated_data)
            return jsonify(student)
    return jsonify({"error": "Student not found"}), 404

# 5. Delete a student
@app.route('/students/<int:student_id>', methods=['DELETE'])
def delete_student(student_id):
    for student in students:
        if student['id'] == student_id:
            students.remove(student)
            return jsonify({"message": "Student deleted"})
    return jsonify({"error": "Student not found"}), 404

if __name__ == '__main__':
```

```
app.run(debug=True)
```

Example Requests and Responses:

S. No.	Operation	Request	Response
1	Get all students	GET http://localhost:5000/students	<pre>[{ "id": 1, "name": "Alice", "course": "Computer Science" }, { "id": 2, "name": "Bob", "course": "Mathematics" }]</pre>
2	Get single student	GET http://localhost:5000/students/1	<pre>{ "id": 1, "name": "Alice", "course": "Computer Science" }</pre>
3	Add a new student	POST http://localhost:5000/students Content-Type: application/json { "id": 3, "name": "Charlie", "course": "Physics" }	<pre>{ "message": "Student added successfully" }</pre>
4	Update a student	PUT http://localhost:5000/students/3 Content-Type: application/json { "course": "Data Science" }	<pre>{ "id": 3, "name": "Charlie", "course": "Data Science" }</pre>
5	Delete a student	DELETE http://localhost:5000/students/3	<pre>{ "message": "Student deleted" }</pre>

6. Limitations of using lists:

- Data is lost when the server restarts.
- No persistence or querying.
- Difficult to handle large datasets.

Using SQLite with Flask

```
import sqlite3
from flask import g

DATABASE = 'books.db'
```

```
def get_db():
    db = getattr(g, '_database', None)
    if db is None:
        db = g._database = sqlite3.connect(DATABASE)
    return db

@app.route('/db/books', methods=['GET'])
def get_books_db():
    cursor = get_db().cursor()
    cursor.execute("SELECT id, title FROM books")
    rows = cursor.fetchall()
    books_list = [{"id": row[0], "title": row[1]} for row in rows]
    return jsonify(books_list)
```

This endpoint fetches all books from the books table in SQLite. Assume the database (books.db) contains the following data:

Id	Title
1	Python Basics
2	Flask for Beginners
3	Machine Learning in Action

Request: GET <http://localhost:5000/db/books>

The response:

```
[
  {
    "id": 1,
    "title": "Python Basics"
  },
  {
    "id": 2,
    "title": "Flask for Beginners"
  },
  {
    "id": 3,
    "title": "Machine Learning in Action"
  }
]
```

This response is generated by converting the SQL query results into a list of dictionaries, which Flask then serializes into JSON.

Advantages:

- Data is persistent.
- SQL queries allow filtering, searching, and joining.
- More scalable and realistic for production APIs.

10.8 REFERENCES/FURTHER READINGS

- R. T. Fielding, Architectural Styles and the Design of Network-based Software Architectures, Ph.D. Dissertation, University of California, Irvine, 2000.
- Flask Web Development: Developing Web Applications with Python by D. Grinberg, 2nd ed., O'Reilly Media, 2018.
- FastAPI Documentation by S. Tiangolo, Available at: <https://fastapi.tiangolo.com>
- Python Software Foundation, Python Official Documentation. Available at: <https://docs.python.org/3/>
- REST API Design Rulebook: Designing Consistent RESTful Web Service Interfaces, by M. Masse, O'Reilly Media, 2011.
- M. Hussain and M. M. Khan, "Performance Comparison of Web Development Frameworks: Django, Flask and FastAPI," Journal of Computer and Communications, vol. 9, no. 10, pp. 1–10, 2021.



ignou
THE PEOPLE'S
UNIVERSITY